# ISTIO

Authen & Author : every micro service needs to communicate with each other that requires A&A

While communicating the services, that will share the data (adding personal details like sharing credit card info etc ...)which is not encrypted.

It's an Infra layer that manages a service to service communication within a micro services architecture. It improves the security, Traffic management, Observability

Before service mesh companies used nginx reverse proxy or load balancers as a service to service communication. But it has security issues. That's why we are using service mesh now.

So to implement istio we need to use side car containers in our pods.

The best side car for istio is Envoy

Because Envoy will helps us in load balancing, data security, Log Management

Create a kops cluster **(master: m7i-flex.large & worker (2) : c7i-flex.large)**

Install istio

curl -L https://istio.io/downloadIstio | sh -

cd istio-1.28.1

export PATH=$PWD/bin:$PATH

Labels a NS : kubectl label namespace default istio-injection=enabled

INSATLL ISTIO CTL : istioctl install --set profile=demo -y

Creare resources (pods and services )

kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml

EXPOSE THE SVC : kubectl patch svc productpage -p '{"spec": {"type": "LoadBalancer"}}'

Get EXTERNAL IP: kubectl get svc

Paste the url:9080/productpage to see the app o/p

**Enable Observability**

**Deploy addons:**

kubectl apply -f samples/addons

#enable LB

# Expose Kiali Dashboard

kubectl patch svc kiali -n istio-system -p '{"spec": {"type": "LoadBalancer"}}'

# Expose Grafana Dashboard

kubectl patch svc grafana -n istio-system -p '{"spec": {"type": "LoadBalancer"}}'

# Expose Prometheus Metrics

kubectl patch svc prometheus -n istio-system -p '{"spec": {"type": "LoadBalancer"}}'

# Expose Jaeger Tracing (via "tracing" service)

kubectl patch svc tracing -n istio-system -p '{"spec": {"type": "LoadBalancer"}}'

**Create a destination rule :**

apiVersion: networking.istio.io/v1beta1

kind: DestinationRule

metadata:

```
 name: reviews
spec:
 host: reviews
 subsets:
 - name: v1
 labels:
 version: v1
 - name: v2
 labels:
 version: v2
 - name: v3
 labels:
 version: v3
```

Create a virtual Service :

```
---

apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
 name: reviews
spec:
 hosts:
 - reviews
 http:
```

- route:

- destination:

host: reviews

subset: v1

weight: 100

- destination:

host: reviews

subset: v3

weight: 0

- destination:

host: reviews

subset: v2

weight: 0

Kubectl apply

Now you can see the reviews are in red colour

Next change the subset to v1 to 100% then you will get reviews